UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Autorización de documentos tributarios mediante una API

Programación orientada a objetos / C# / HTTP

William René Toledo Corado

Carné: 202210198

Sección B

Introducción a la Programación y Computación 2

Escuela de Ciencias y Sistemas / Facultad de Ingeniería

Proyecto 3 / Segundo semestre 2026

Guatemala, 22 de octubre de 2026

Ensayo técnico y anexos de diseño

Cinco páginas de contenido. Diagramas y referencias en anexos.

Revisar su adecuación a la plantilla de ensayo del curso antes de entregar.

1. Introducción: la autorización como un proceso verificable

Una solicitud de autorización tributaria combina identidad, tiempo, dinero y trazabilidad. La presencia de los campos de una factura no demuestra que el documento sea coherente: sus contribuyentes deben estar registrados, sus identificadores deben superar una comprobación matemática y los montos deben corresponder entre sí. El problema planteado para el Proyecto 3 de IPC2 consiste en organizar estas verificaciones como un servicio HTTP que pueda utilizarse desde una aplicación web o desde otro cliente. La propuesta sostiene que separar la presentación, las reglas de negocio y el almacenamiento permite examinar cada decisión y comprobarla con entradas reproducibles [1].

El Registro Tributario Unificado constituye el punto de partida del proceso. Un mensaje JSON permite incorporar contribuyentes nuevos o actualizar el nombre asociado a un NIT existente. Una entrada con verificador inválido no modifica el registro. Después, los mensajes de facturación solicitan autorizaciones e incluyen fecha, referencia, emisor, receptor, valor, IVA y total. La respuesta identifica las autorizaciones emitidas y los motivos de rechazo. Esta separación evita confundir una actualización de identidad con la emisión de un documento y permite cargar primero el RTU para resolver las referencias de las facturas posteriores.

El alcance funcional abarca también la consulta de resultados. No basta con producir un código: deben conservarse los datos necesarios para reconstruir las cantidades recibidas, las categorías de error y las sumas autorizadas. Por ello, el sistema ofrece consultas de facturas emitidas o recibidas por NIT, un resumen de IVA por fecha y contribuyente, valores autorizados por rango y un reporte diario de errores. Las gráficas facilitan la comparación y los PDF permiten obtener una representación independiente de la pantalla. Los totales financieros se calculan únicamente sobre documentos autorizados; los intentos rechazados participan en las estadísticas de validación.

Existe una diferencia esencial entre el momento del documento y el momento de su autorización. Una factura puede contener una fecha anterior a la del envío. El código utiliza el día en que el servicio la autoriza en Guatemala, mientras que las consultas utilizan la fecha extraída de TIEMPO. Conservar ambos momentos permite explicar un código actual asociado a una factura histórica sin alterar los archivos de entrada. El funcionamiento se limita a un simulador académico local: no se conecta a servicios reales de la SAT ni presenta las autorizaciones generadas como documentos fiscales oficiales.

2. Diseño orientado a objetos y comunicación entre componentes

La solución contiene tres proyectos con responsabilidades diferenciadas. SatDte.Web implementa la presentación con Razor Pages; SatDte.Api publica las operaciones HTTP; SatDte.Core reúne las entidades, validaciones, servicios, consultas y mecanismos de persistencia. Razor Pages relaciona cada pantalla con su PageModel, donde se reciben las entradas y se solicita información al servicio [2]. El frontend utiliza HttpClient y no lee el archivo de datos. Esta decisión reproduce la arquitectura del enunciado y permite verificar la API desde Postman sin depender de la interfaz gráfica.

Las entidades principales son Contribuyente y Factura. La primera relaciona el NIT con el nombre vigente del RTU. La segunda conserva los participantes, la fecha documental, los montos, el momento de recepción, los errores encontrados y el código de aprobación cuando existe. EstadoSistema agrupa los contribuyentes, los intentos de facturación, los descartes y los correlativos diarios. Un diccionario facilita la búsqueda de contribuyentes por NIT; una lista conserva los intentos en orden de procesamiento; un conjunto temporal permite detectar referencias autorizadas sin recorrer todas las facturas por cada comparación.

Las abstracciones IRepositorio e IReloj reducen la dependencia de los servicios respecto de detalles externos. ServicioDte solicita al repositorio una operación de lectura o actualización, sin administrar directamente los archivos. ServicioConsultas obtiene una instantánea y construye las agrupaciones necesarias para cada petición. IReloj proporciona el momento actual; RelojGuatemala lo expresa con desplazamiento UTC-6. Durante las pruebas se utiliza un reloj controlado que permite simular el paso de medianoche y comprobar el reinicio del correlativo sin esperar al día siguiente. La encapsulación protege estas decisiones detrás de operaciones explícitas.

Las rutas HTTP son una capa de adaptación al dominio. Reciben JSON o parámetros, invocan los servicios y devuelven JSON, SVG o PDF. ASP.NET Core permite declarar estos endpoints mediante Minimal APIs [3]. En este proyecto, las reglas no se concentran en las rutas: permanecen en clases de servicio que también se prueban directamente. Las respuestas conservan las claves exigidas para RTU y el contenedor resultadoAutorizaciones para facturación. Los contadores adicionales y los detalles explican los errores requeridos por los casos de prueba sin cambiar los campos obligatorios del contrato.

El procesamiento distingue errores de la solicitud completa y errores de una fila. Un JSON malformado, un objeto suelto, un arreglo vacío o un lote mixto reciben una respuesta 400 antes de modificar el estado. Una factura correctamente estructurada puede fallar en varias reglas de negocio y se registra como un intento rechazado. La Web muestra la respuesta de cada archivo, mantiene el orden de envío y puede continuar con los archivos siguientes. Así, la interacción ayuda a localizar el problema sin ocultar la respuesta original del servicio.

3. Validación de identidad, fechas y cantidades monetarias

El NIT se representa como texto, porque una base de veinte dígitos puede superar los límites de un entero y porque los ceros iniciales deben conservarse. Una expresión regular verifica que existan entre uno y veinte dígitos de base seguidos de un verificador numérico o K. Se normaliza la letra minúscula para aceptar el caso de borde incluido en el ZIP. El algoritmo separa el verificador, recorre la base de derecha a izquierda y multiplica sus dígitos por pesos que comienzan en dos. La suma se transforma mediante módulo once; el resultado diez equivale a K. Después se compara con el carácter recibido.

Esta comprobación precede a la búsqueda en el RTU. Un NIT que no supera el verificador produce un error de identidad inválida. Si lo supera y no aparece en el registro, produce un error de contribuyente inexistente. Aplicar ese orden evita contar el mismo problema en ambas categorías. El ejemplo 5123456 del documento no supera el algoritmo; el sistema no incorpora una excepción para aceptarlo. Las pruebas proporcionadas incluyen ese identificador como inválido y utilizan 5123453 como su contraparte válida. El criterio aplicado queda documentado para facilitar su revisión frente a cualquier aclaración del curso.

La fecha se obtiene de TIEMPO mediante una expresión regular con grupos para día, mes, año, hora y minuto. La expresión localiza la información relevante dentro del texto y descarta nombres de lugares o el sufijo decorativo. Después, DateTime verifica que la combinación exista en el calendario. Esta segunda etapa rechaza, por ejemplo, el 29 de febrero de 2025 y acepta el de 2024. También rechaza las 24:00 y las fechas sin hora. La tolerancia de uno o dos dígitos en día, mes y hora responde a los casos de borde suministrados; no convierte una fecha imposible en una fecha válida.

Para los montos se utiliza decimal, adecuado cuando interesa la precisión de las posiciones decimales, como ocurre con cantidades monetarias [4]. El IVA esperado es el producto del valor por 0.12, redondeado a dos decimales con MidpointRounding.AwayFromZero. El total se compara con el valor más el IVA recibido. Esta distinción explica por qué una factura con valor 100, IVA 13 y total 113 registra error de IVA, pero no un segundo error de suma. Los valores no numéricos, negativos o con fracciones adicionales se clasifican como problemas de formato antes de evaluar las reglas tributarias.

La referencia tiene de uno a cuarenta caracteres y se compara exactamente. Una autorización reserva la referencia; una fila rechazada permite corregirla y reenviarla. Dentro de un mismo lote, una referencia aprobada se incorpora inmediatamente al conjunto de referencias utilizadas. La validación acumula los errores de cada factura, en vez de detenerse en el primero. De esta manera, el emisor puede corregir todos los problemas detectados y las estadísticas mantienen contadores independientes sin sumar una factura varias veces al total de rechazadas.

4. Persistencia, correlativos y construcción de reportes

La persistencia debe garantizar que una respuesta de aprobación corresponda a un estado guardado. RepositorioJson mantiene una copia del estado confirmado y ejecuta cada actualización sobre una copia de trabajo. Dentro de un bloqueo, el servicio aplica las verificaciones, registra los intentos y asigna los códigos. Después, el repositorio serializa la copia a un archivo temporal, vacía su contenido a disco y reemplaza el archivo definitivo. Solo al completar esa escritura se adopta el nuevo estado en memoria. Si la operación falla antes de confirmar, la respuesta de éxito no se emite y la copia previa continúa disponible.

El bloqueo cubre la operación completa, incluida la asignación del correlativo. Sin esa exclusión, dos solicitudes podrían leer el mismo último valor y emitir códigos repetidos. El estado conserva un contador por día; la fecha de autorización en Guatemala selecciona el contador que se incrementa. El código combina ocho posiciones de fecha con ocho posiciones de correlativo. Los rechazos y descartes no consumen números. Un archivo abierto con FileShare.None impide que otra instancia del repositorio escriba sobre el mismo estado. La propuesta contempla un proceso API por archivo, no una infraestructura con escritores distribuidos.

Al reiniciar, la API carga el archivo existente y continúa sus contadores. Un archivo ilegible o de versión incompatible detiene el arranque; no se sustituye silenciosamente por un estado vacío. Esta conducta permite distinguir una instalación sin datos de un problema de integridad que requiere revisión. La operación Inicializar es explícita y elimina RTU, facturas, descartes y correlativos. La pantalla muestra una confirmación antes de enviarla, ya que devolver el simulador al estado inicial impide reconstruir sus datos a partir del archivo activo. Para repetir las pruebas se deben cargar de nuevo los originales.

Las consultas obtienen una instantánea que no cambia mientras se calcula el reporte. Para un contribuyente se filtran las autorizadas por NIT, sentido y rango inclusivo; después se agrupan por fecha y se calculan cantidades y montos. El resumen de IVA reúne los NIT que participaron en un día y suma por separado sus movimientos emitidos y recibidos. En el rango general se selecciona el valor sin IVA o el total de cada factura. El reporte de errores incluye los intentos rechazados y cuenta las categorías de cada día, junto con emisores y receptores distintos.

Los descartes de formato se conservan aparte porque no siempre proporcionan una fecha documental utilizable. Se informa su posición, referencia, motivo y momento de recepción, pero no se les inventa una fecha para incorporarlos al reporte diario. Las gráficas SVG presentan las mismas agrupaciones que las tablas. Los PDF se construyen en C# con texto seleccionable y paginación, sin servicios externos. La reutilización de los resultados de consulta para JSON, gráfica y PDF mantiene un único criterio de cálculo y permite contrastar la pantalla con la información devuelta directamente por HTTP.

5. Evaluación de la solución y conclusiones

La evaluación utiliza los archivos del ZIP proporcionado sin modificarlos y contrasta sus resultados con el LEEME. El primer mensaje del RTU produce nueve incorporaciones, una actualización y tres NIT inválidos. Las facturas de marzo generan cuatro autorizaciones. El conjunto de errores genera tres aprobaciones, diez facturas rechazadas y un descarte de formato; las categorías se verifican de manera independiente. Los casos de borde producen once aprobaciones y dos descartes. La carga de volumen produce cien autorizaciones adicionales. Estos resultados permiten comprobar el comportamiento solicitado con datos que no se originan en los servicios de la solución.

Las consultas también se contrastan con cantidades concretas. Para el NIT 999999K, las emitidas del primero al dos de marzo de 2025 totalizan Q 9,672.32 y las recibidas Q 9,258.06. El rango general alcanza Q 18,930.38 con IVA y Q 16,902.13 sin IVA. En el resumen del dos de marzo, ese contribuyente presenta IVA emitido de Q 494.06 y recibido de Q 392.20. La matriz del reporte de errores se compara completa con las once columnas del LEEME. Esto verifica las agrupaciones y las sumas, además de comprobar que la API acepta los mensajes.

Las pruebas del núcleo incluyen concurrencia, referencias compartidas, cambio de día, reapertura del repositorio y reinicialización. Las pruebas HTTP agregan JSON malformado, códigos de estado, descargas PDF, SVG y un reinicio real del proceso. La ejecución registrada obtuvo veinticuatro comprobaciones correctas del núcleo, treinta comprobaciones HTTP correctas y veinte comprobaciones del frontend correctas. La revisión del navegador verificó la selección múltiple, la vista previa, el envío secuencial y la consulta de resultados. Durante esa revisión se corrigió una validación de filtros que impedía consultar IVA sin aportar un NIT. La evidencia se incluye separada del ensayo para permitir su reproducción.

La separación de responsabilidades permite relacionar un resultado observable con una regla y una prueba. El diseño orientado a objetos encapsula la persistencia y el reloj; las expresiones regulares extraen la información relevante; el uso de decimal mantiene un criterio monetario explícito. La escritura transaccional y la exclusión de solicitudes simultáneas sostienen la unicidad de los códigos dentro de las condiciones locales de ejecución. La solución ofrece, por tanto, un flujo completo de registro, validación, autorización y análisis, con respuestas que explican tanto las aprobaciones como los errores.

Las limitaciones también delimitan el resultado: el almacenamiento usa un archivo local y un solo escritor, la aplicación está configurada para localhost y no integra una certificación fiscal real. La elaboración del paquete no acredita versiones semanales previas ni realiza una publicación en GitHub o una entrega en UEDI. Esas actividades requieren la cuenta y el historial real del estudiante. La plantilla específica del ensayo del curso no fue adjuntada, por lo que corresponde revisar el formato final. Los anexos presentan el modelo de clases, las actividades principales y el despliegue que corresponden al código entregado.

Anexo A. Modelo de clases

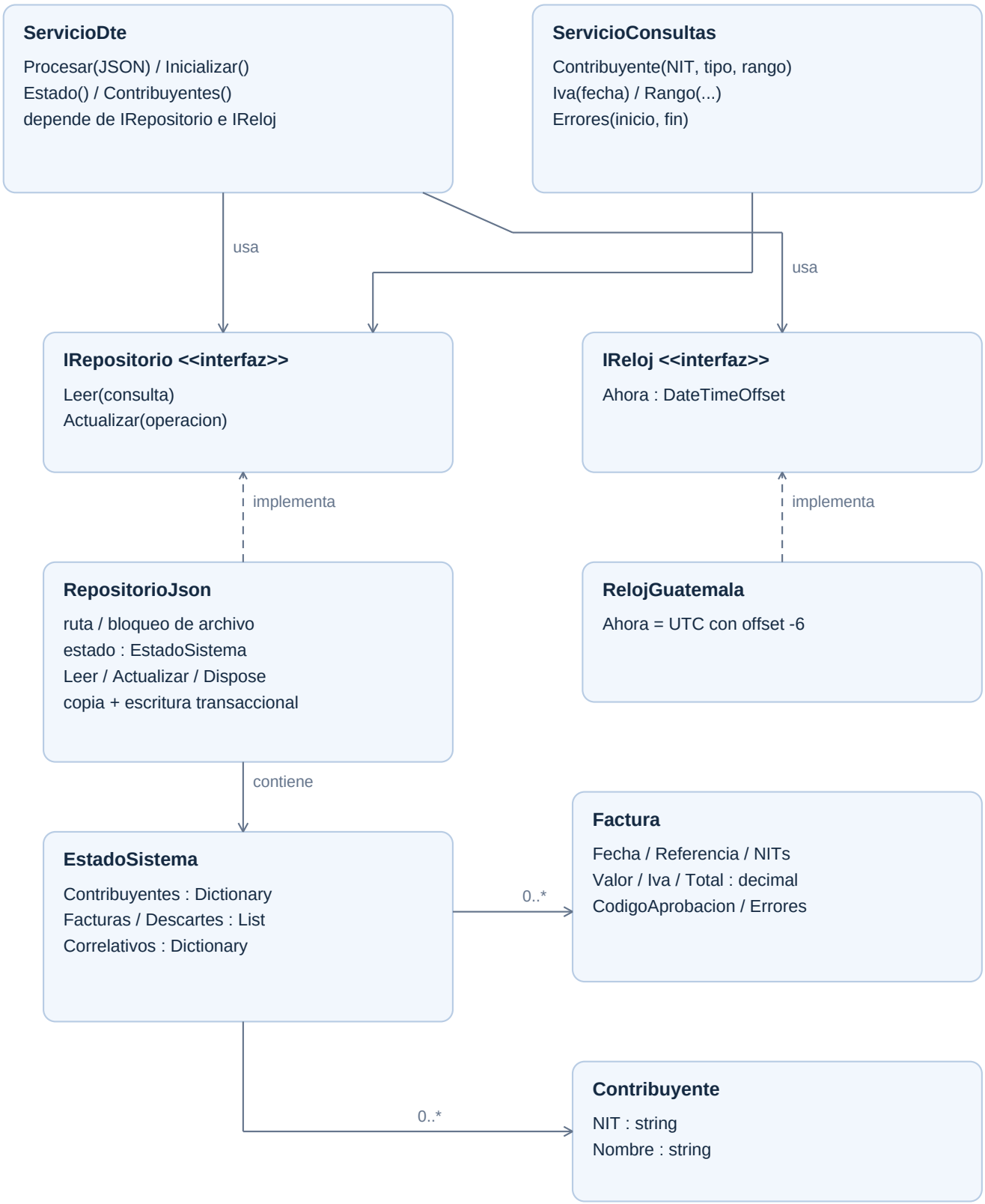


Figura A1. Clases e interfaces principales del núcleo y relaciones de dependencia.

Anexo B. Actividad de procesamiento

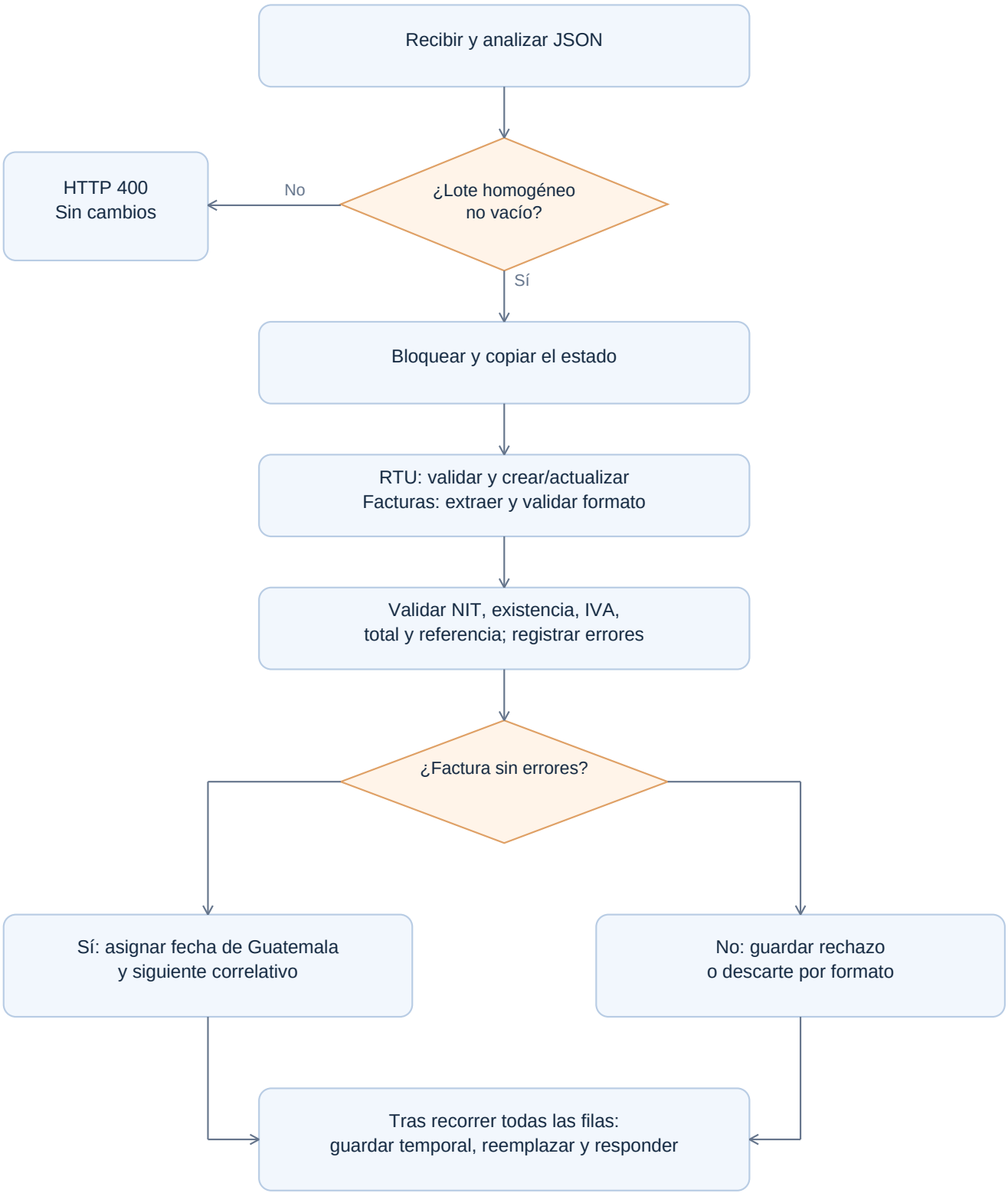
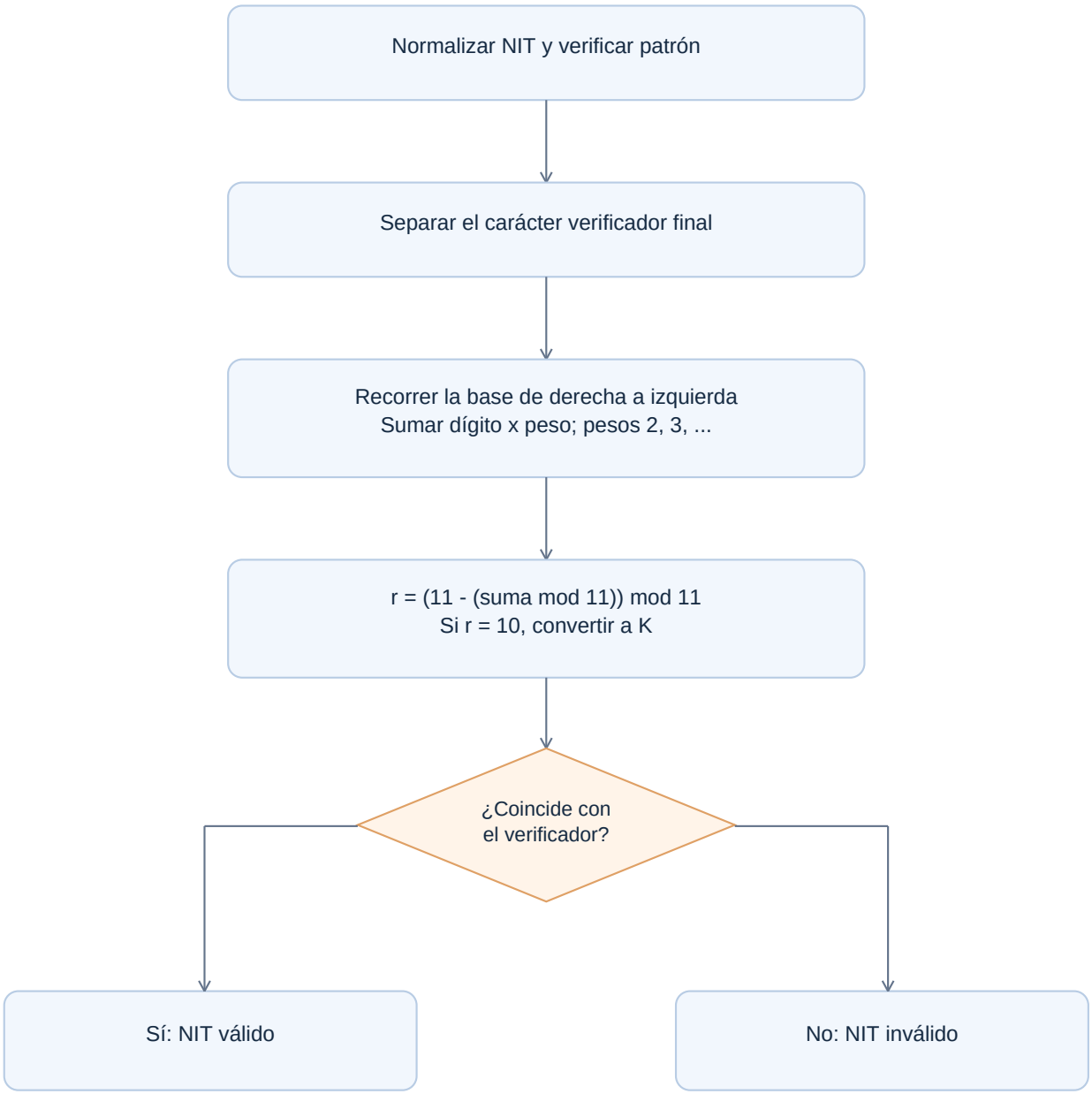


Figura B1. Validación, autorización y confirmación de un lote. Los descartes de formato omiten las reglas de negocio; un lote RTU crea o actualiza solo los NIT válidos.

Anexo C. Actividades de validación y consulta



Actividad de consulta

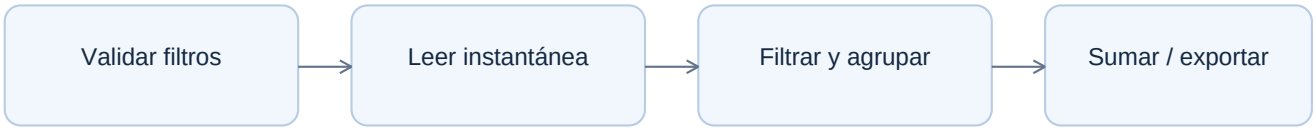


Figura C1. Algoritmo módulo once para NIT y secuencia de construcción de consultas.

Anexo D. Despliegue y referencias

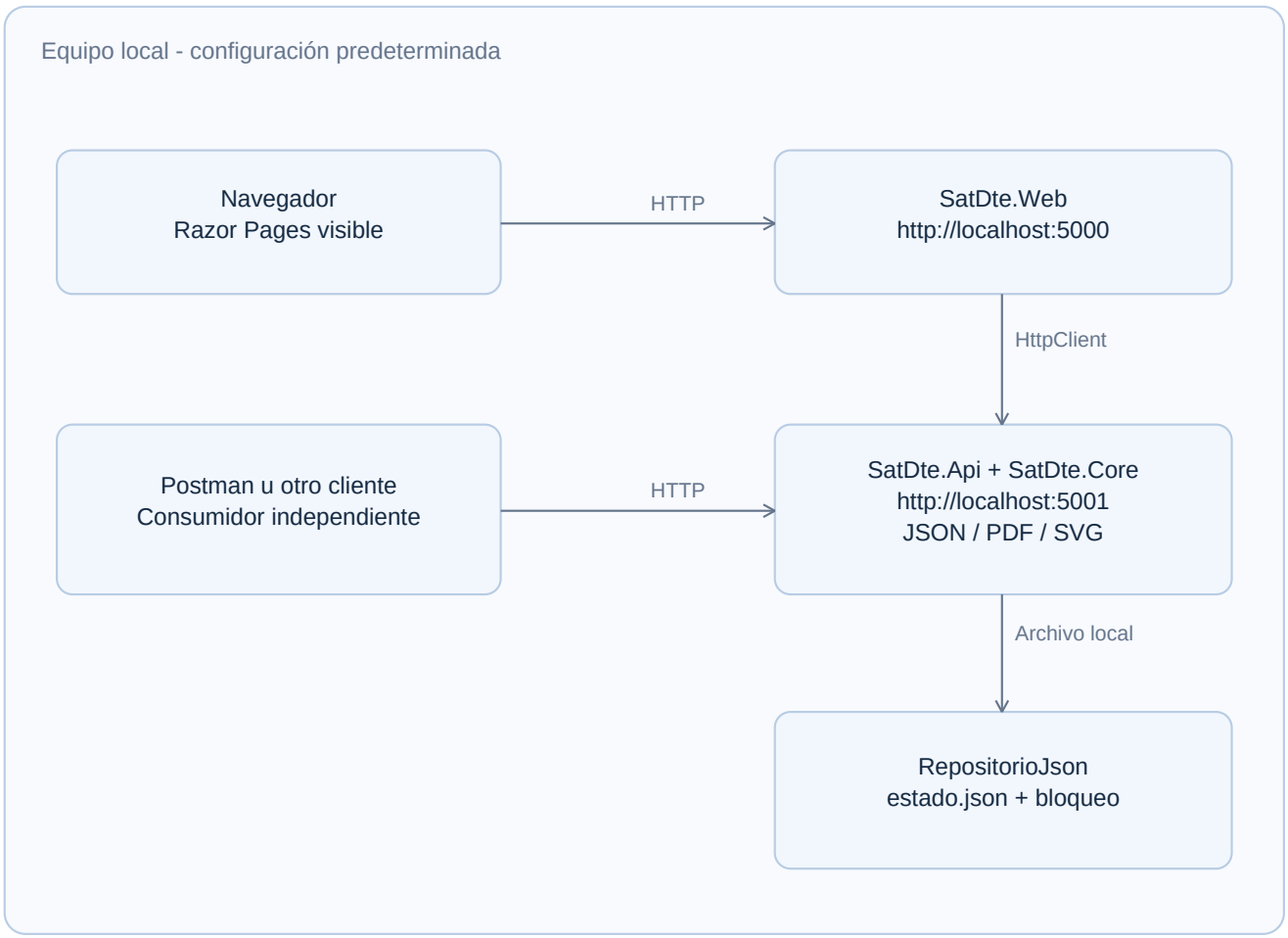


Figura D1. Dos procesos HTTP y un almacenamiento local; la Web consume la API.

Referencias

[1] USAC, IPC2. Proyecto 3, segundo semestre 2026, versión 1. Enunciado y archivos de pruebas proporcionados por el usuario.

[2] Microsoft Learn. Razor Pages architecture and concepts in ASP.NET Core. https://learn.microsoft.com/en-us/aspnet/core/razor-pages/?view=aspnetcore-10.0

[3] Microsoft Learn. APIs overview. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis?view=aspnetcore-10.0

[4] Microsoft Learn. Floating-point numeric types: C# reference. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types